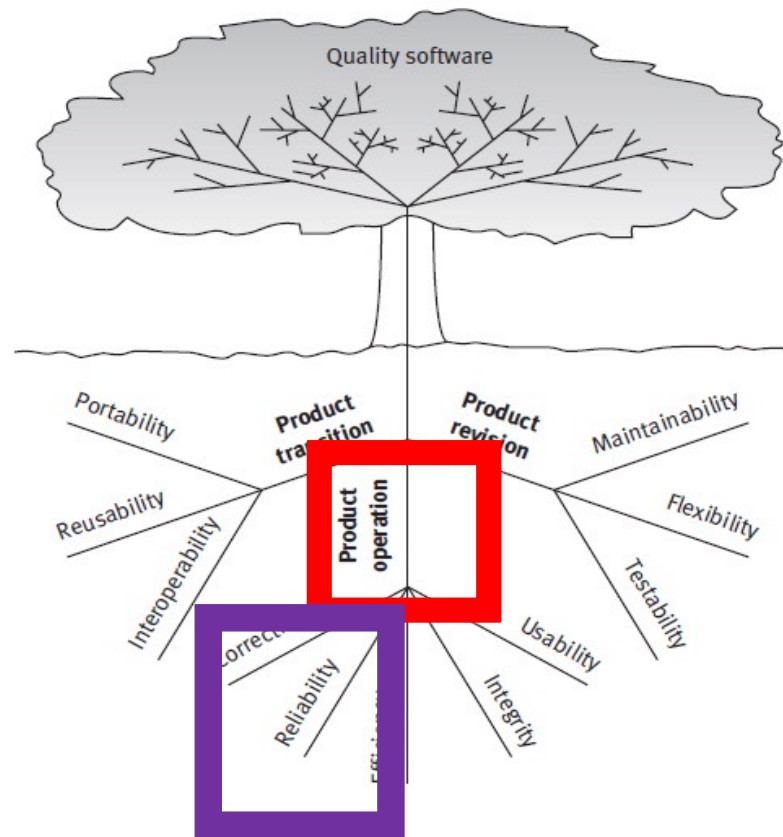


Course 8

Reliability, Reusability, interoperability



Reliability



Reliability



Definition

in engineering: *the ability of a product or component to continue to perform its intended role over a period of time to pre-defined conditions*

software systems: *is measured in terms of the **mean time between failures**, the mean time to repair, the **mean time to recover**, the **probability of failure** and the general availability of the system.*



Aspects:

Maturity – absence of software faults that lead to failures

Robustness (Fault tolerance) – performance to specification despite some faults

Recoverability – operation after a failure

Reliability issues

Input
problems

Output
problems

Logic
problems

Computation
problems

Data-handling
problems

Fault
tolerance

Input problems



DON'T accept wrong input (out-of range values)



Accept all possible correct input



Reject unreasonable input – careful



Consider missing input elements

Output problems

- Incomplete or missing output
- Correct results at the wrong time
- Wrong format

Adding an element to a program and forgetting to update output routines

Adding an element to a program and forgetting to update output routines

Adding an element to a program and forgetting to update output routines

Logic problems

- Associated with program's control flow
- Affect operations and data
- Difficult to detect and locate
- Typical:
 - Off-by-one errors
 - Neglecting extreme conditions
 - Forgotten cases, condition tests or steps
 - Misinterpretation of specification

- Missing processing or counting the first or last element of a range

Values outside range

Computation problems

- Incorrect algorithm or computation
- Uninitialized variables
- Referencing NULL
- Abusing type system

Data-handling problems

- Incorrect data initialization
- Referencing wrong data variable
- Out-of-bounds values
- Inconsistent data – representation – add, update, delete

Fault tolerance

- **Fault** = software or hardware problem in a component of a system under examination
- Fault – leads to a system failure: behavior deviated from specification requirement
- Fault-tolerance system:
 - detect and diagnose the fault
 - report/ mask/ compensate it

=> Avoid failure

Reliability McCall model

Definition: *extend to which a program can be expected to perform its intended function with required precision*



Includes:

Availability

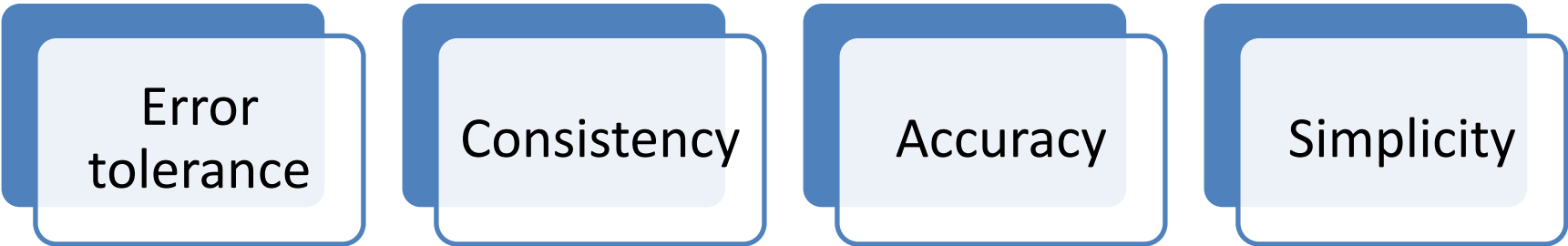
Accuracy

Robustness

Precision

Tolerance

Criteria



Error
tolerance

Consistency

Accuracy

Simplicity

Evaluation (see McCall.pdf)

- Based on criteria:
 1. Consistency: procedure + data consistency
 2. Accuracy
 3. Error tolerance: error tolerance, input data, recoverable computational failures, recoverable hardware faults, device status condition
 4. Simplicity: design structure, programming paradigm (~~structured~~), data and control flow complexity, code simplicity

Tools for reliability evaluation

Explanation of terms

- **Fault Tree Analysis** – identifies and ranks combinations of events that can lead to system failure
- **Markov analysis** – calculates reliability based on simulation
- **Binary Decision Diagram** – quick decision processing, based on statistical distribution of failure probabilities, computes exact measure of event importance,...

Fault Tree Analysis (FTA)

- deductive failure analysis based on Boolean logic

Deductive $\Rightarrow$ **backwardly** deduce the causes of the event

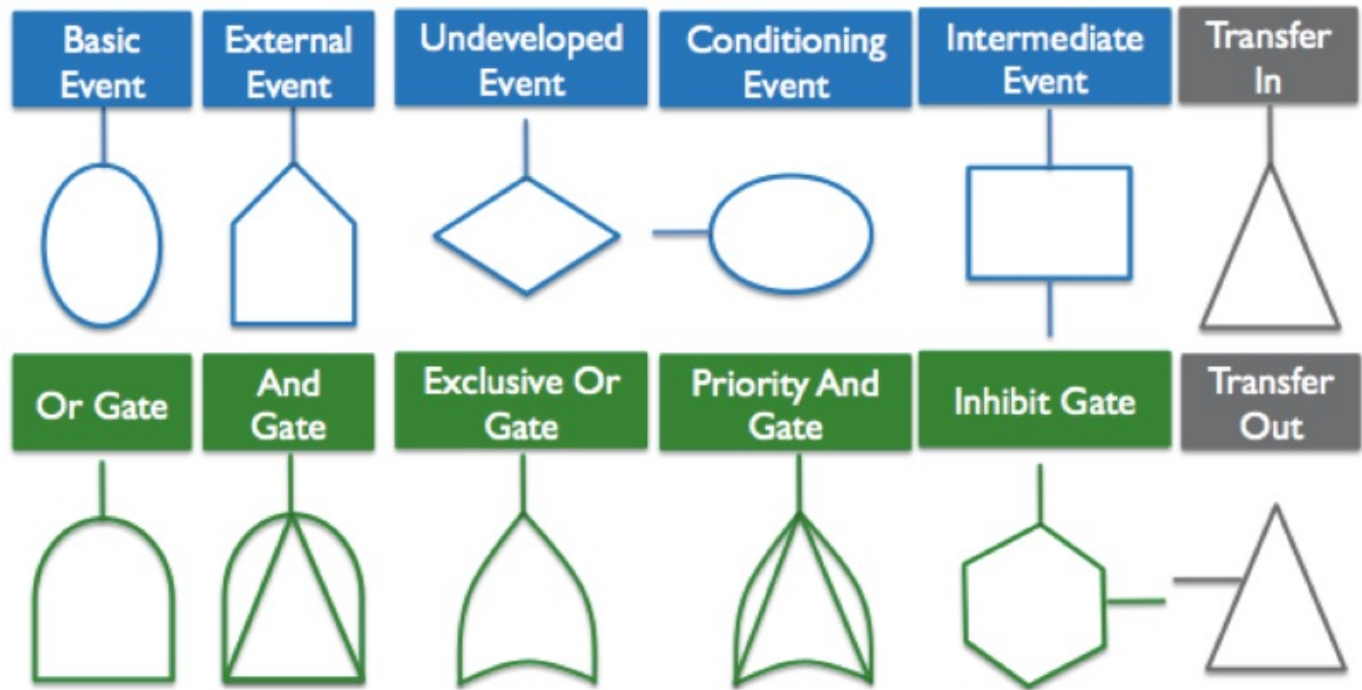
- 1962 – Bell Laboratories – aviation
- Used in reliability and safety analysis
- Used in SE – debugging

Fault Tree Analysis (FTA)

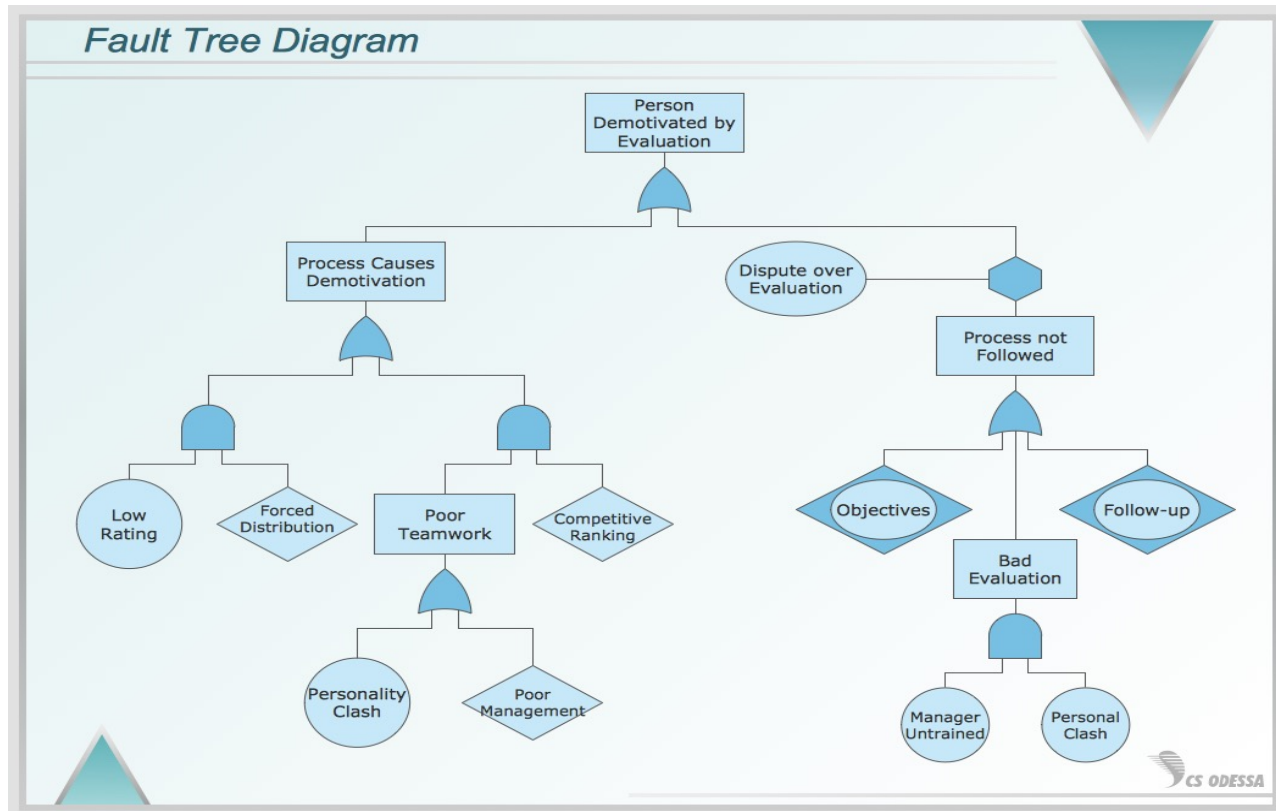
- Analyze an undesired state of the system – combine series of lower-level events:
 - 1) Defined undesired event
 - 2) The event is resolved into its immediate causes
 - 3) This resolution of events continues until basic causes are identified
 - 4) A logical diagram called a fault tree is constructed showing the logical event relationships

Fault Tree Analysis (FTA)

- Graphical symbols:
 - Event
 - Gate
 - Transfer



FTA- example



try

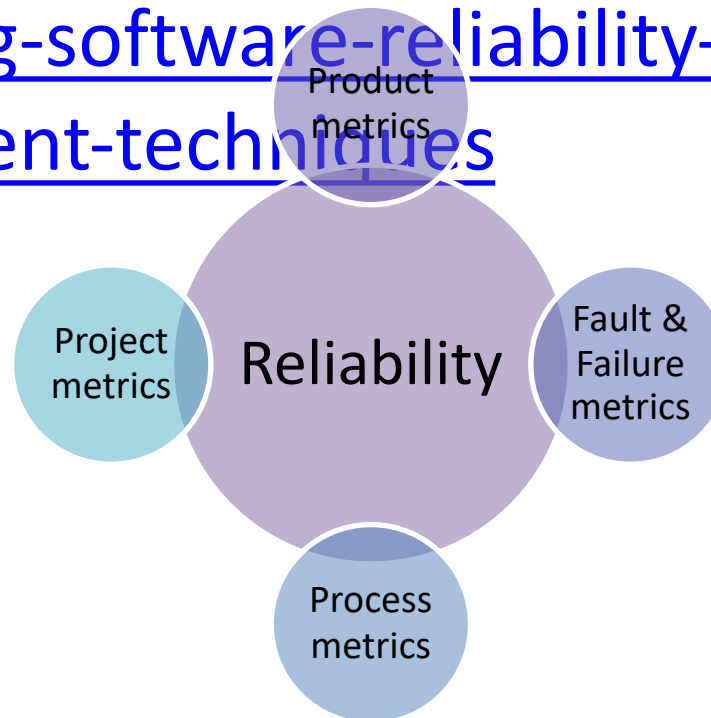
- <http://www.fault-tree-analysis-software.com>
- <https://www.smartdraw.com/fault-tree/>

OTHER RELIABILITY ESTIMATION METHODS (*LIGHTWEIGHT*)



New 2021

- <https://www.javatpoint.com/software-engineering-software-reliability-measurement-techniques>



Fault & Failure Metrics:

quantitatively expressed the reliability of the software product

- Mean Time to Failure (MTTF)
- Mean Time to Repair (MTTR)
- Mean Time Between Failure (MTBR)
- Rate of occurrence of failure (ROCOF)
- Probability of Failure on Demand (POFOD)
- Availability (AVAIL)

Estimate Reliability - SonarQube

- **Reliability Rating** (reliability_rating)

A = 0 Bugs

B = at least 1 Minor Bug

C = at least 1 Major Bug

D = at least 1 Critical Bug

E = at least 1 Blocker Bug

Estimate Reliability - MetricsReloaded

$$QCP \text{ RLBT } Y = N + (2 * NEST) + (3 * V(g)) + \text{BRANCH} + \text{CONTROL} + \text{EXEC}$$

where:

- N = Halstead Length metric for a method
- NEST = maximum nesting depth of each method
- V(g) = cyclomatic complexity
- BRANCH = total number of non-structured branch statements in each method
- CONTROL = total number of control statements in each method
- EXEC = the total number of executable statements in each method

Estimate Reliability - metrics

- WMC
- CBO
- LCOM5
- RFC - Response for a Class
(number of local (i.e. not inherited) methods in the class plus the number of directly invoked other methods by its methods or attribute initializations)
- LOC - Lines of Code



<https://www.iso.org/obp/ui/#iso:std:iso-iec:5055:ed-1:v1:en> based on
CWE <https://cwe.mitre.org/>

<https://www.iso.org/standard/80623.html>

RELIABILITY – ISO:IEC 5055/2021

Reusability

- Definition: *Extent to which a program can be used in other applications - **related to the packaging and scope of the functions** that programs perform.*
- Impact:
 - Measured: design + implementation
 - Realized: transition

Reusability

```
graph TD; Reusability[Reusability] --- Generality[Generality]; Reusability --- Modularity[Modularity]; Reusability --- Software[Software system independence]; Reusability --- Machine[Machine independence]; Reusability --- Self[Self-descriptiveness];
```

Generality

Modularity

Software system
independence

Machine
independence

Self-
descriptiveness

Reuse what?

- Architecture
- Source code
- Data
- Design
- Documentation
- Templates
- GUI
- Requirements
- Test cases



language



paradigm



methodology

1. Paradigms that facilitate reuse?
2. Languages that facilitate reuse? How?
3. Methodologies that facilitate reuse?

Reusability

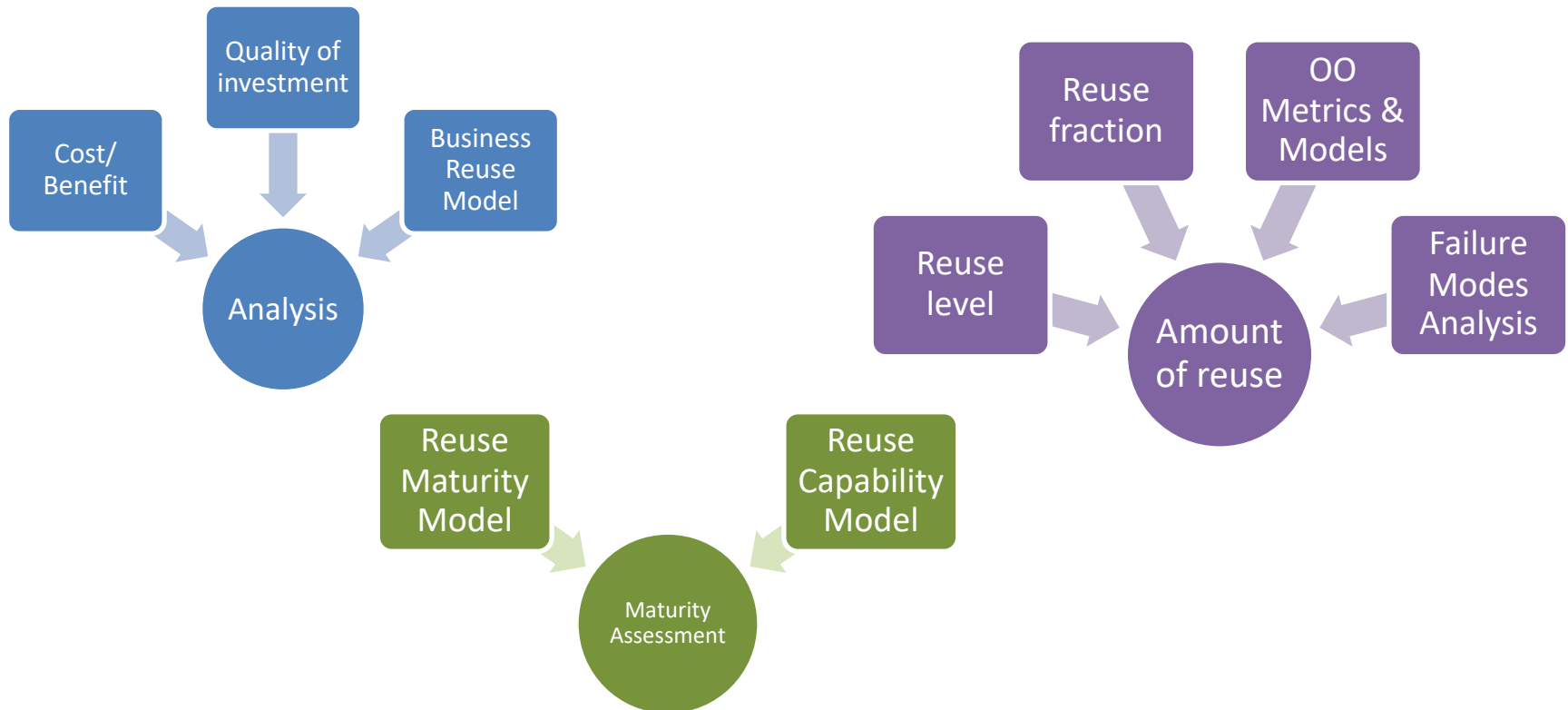
- Paradigms:
 - OOP
 - Component based programming
- Languages: OOL
 - C++
 - Java
- Methodology
 - CBSD
 - MDD

STL, templates,
multiple inheritance

interfaces,
RPC

Reuse: metrics & models

[W. Frakes, C. Terry – Software Reuse: Metrics and Models, ACM Computing Surveys 28(2), pp. 415-435, 1996]



Further reading:

- [https://www.researchgate.net/publication/282867676 Reusability Metrics of Software Components Survey](https://www.researchgate.net/publication/282867676_Reusability_Metrics_of_Software_Components_Survey)
- <http://jeffreypoulin.info/Papers/ICSR94/icsr94.pdf>

Study on Reusability Models

[Andreea Lung (IS stud.), S. Motogna, V. Petrascu)]

Paper describing the model	Computation	Excluding criteria			Metrics	Reusability models		
Reusability Index: A Measure for Assessing Software Assets Reusability (Ampatzoglou and Stamatia, 2018)	Backwards Linear Regression	Calculation only at component and asset level	Factors			PDS	Taibi	QMOOD
Measuring the Reusability of Software Components using Static Analysis Metrics and Reuse Rate Information (Papamichail et al., 2019)	Hierarchical approach, polynomial regression	accepted	Complexity	Short name	Meaning			
A Reusability Evaluation Model for OO-Based Software Components (Sandhu and Singh, 2008)	Neuro-fuzzy inference engine	Only structural quality factors considered		ACC	Average Cyclomatic Complexity		✓	
Estimation of Software Reusability for Component based System using Soft Computing Techniques (Singh et al., 2014)	Neural Networks	Requires implementation of neural network in Matlab	Coupling / Modularity	NLE	Nesting Level	✓		
Empirical Analysis of the Reusability of Object-Oriented Program Code in Open-Source Software (Taibi, 2014)	Based on three quality factors	accepted		WMC	Nesting Level Else-If	✓		
A hierarchical model for object-oriented design quality assessment (Bansiya and Davis, 2002)	Hierarchical, based on empirical and anecdotal info	accepted		CBO	Weighted Methods per Class	✓		
Estimation of Software Reusability: An Engineering Approach (Nair and Selvarani, 2010)	empirical, weighted combination of polynomials	only structural code characteristics		BOI	Coupling Between Objects	✓	✓	✓
				NII	Coupling Between Objects Inverse	✓		
				NOI	Number of Incoming Invocations	✓		
				RFC	Number of Outgoing Invocations	✓		
				LCOM5	Response set For Class	✓		
				ROI	Lack of Cohesion in Methods 5	✓	✓	✓
				CIC	Relevance of Identifiers		✓	
				AD	Correlation Identifiers Comments	✓		
				CD	API Documentation	✓		
				CLOC	Comment Density	✓		
				DLOC	Comment Lines of Code	✓		
				PDA	Documentation Lines of Code	✓		
	Inheritance			DIT	Public Documented API	✓		
	Size			LOC	Depth of Inheritance Tree	✓	✓	
				LLOC	Lines of Code	✓		
				TNA	Logical Lines of Code	✓		
				NOC	Total Number of Attributes	✓		
				NM	Number of Children			✓
				NPM	Number of Methods		✓	
				NG	Number of Public Methods			✓
				TNOS	Number of Getters	✓		
					Total Number of Statements	✓		

Study on Reusability Models

[Andreea Lung (IS stud.), S. Motogna, V. Petrascu)]

- 3 open source projects:

	Mockito	JUnit4	Atmosph.
Lines of code	90,926	47,402	61,841
No of classes	1,970	1,505	899
Versions	605	28	219
Used by	107k	1.8m	134
GitHub stars	8.2k	12.1K	3.5

Findings:

1. Variations appear in earlier versions; reusability becomes stable in later versions
2. Reusability influenced by: complexity, cohesion, coupling, size, inheritance
3. Differences in estimation models

Anyone interested in topic? Mail me

Interoperability

Interoperability

- Definition: *Effort required to couple one system with another*
- Impact:
 - Measured: design
 - Realized:
 - operation
 - transition

Interoperability

```
graph TD; Interoperability[Interoperability] --> Modularity[Modularity]; Interoperability --> Communication[Communication communality]; Interoperability --> Data[Data communality]; Communication --> Protocols[use of standard protocols and interface routines]; Data --> Representations[use of standard data representations];
```

Modularity

Communication
communality

Data
communality

use of
standard
protocols
and interface
routines

use of standard
data
representations

Interoperability issues



Interoperability

Syntactic

- guarantees only that data will pass through a connector properly
- Specified data formats, communication protocols
- Ex. XML, SQL standards

Semantic

- achieved only when components agree on the *meaning* of the data they exchange
- automatically interpret the information exchanged meaningfully and accurately (semantic web, ontology)

Interoperability

- Data interoperability:
 - XML
 - JSON
- Web interop:
 - SOAP
 - Web services
 - microservices

Interoperability metrics

1. Carnegie Mellon:

[Carnegie Mellon University – Measuring System Interoperability, version 1.0]

- Technical compliance measures

Scoreboard:
System – score
(pass/margin/fail)

- Systems interoperability measures

Scoreboard:
System per system –
score

- Operational interoperability measures

Information flow -
systems– score

- Organizational and cultural measures

Interoperability metrics

2. LISI (Levels of Information Systems Interoperability)

[MITRE, C4ISR]

- Level 0: isolated – no connection (manual gateway)
- Level 1: connected – electronic connection; separate data & application (email, FM voice, tactical data links, text files)
- Level 2: functional – minimal common functions; separate data & application (annotated images, maps)
- Level 3: domain – shared data & separate applications (common operational picture)
- Level 4: enterprise – cross-domain information, interactive manipulation, shared data & applications (event-triggered global database update)

LCIM - Levels of Conceptual Interoperability Model

- https://www.researchgate.net/publication/224178883_Architecture_constraints_for_Interoperability_and_composability_in_a_smart_grid/figures?lo=1

Interoperability metrics

3. ATL Use Case - Software Quality Control Tools Interoperability (Bugzilla, Mantis, Excel)

<https://www.eclipse.org/atl/usecases/SoftwareQualityControlToolsInteroperability/>